

Unit 1

Getting Started with ASP.NET Core: Frameworks, Development, and Testing

ASP.NET Core is a modern, open-source, and cross-platform web development framework developed by Microsoft for building web applications, websites, web APIs, and cloud-based services. It provides a structured environment that helps developers create secure, scalable, and high-performance applications that can run on Windows, Linux, and macOS. ASP.NET Core includes application frameworks such as MVC (Model-View-Controller), Razor Pages, and Web API, which help organize application code and simplify development. It also provides utility frameworks such as Dependency Injection, Logging, Configuration Management, and Authentication, which support common development tasks and improve application maintainability. These features make ASP.NET Core suitable for developing everything from small websites to large enterprise systems.

Putting ASP.NET Core in Context

What is an Application Framework?

An application framework is a collection of libraries, tools, and pre-written code that provides a structured way to develop software applications. Instead of writing every component from scratch, developers can use the framework's built-in features to handle common tasks such as routing, security, database access, authentication, and user interface development. This reduces development time and improves application reliability.

<https://www.youtube.com/watch?v=L4fanHl2beA>

1. Understanding the Application Frameworks

Application frameworks provide a ready-made structure for building software applications. Instead of writing everything from scratch, developers use frameworks that contain libraries, tools, and predefined patterns. In ASP.NET Core, application frameworks help create web applications, APIs, and dynamic websites quickly and efficiently. They promote code reuse, easier maintenance, and better organization of projects. Common application frameworks in ASP.NET Core include MVC (Model-View-Controller), Razor Pages, and Web API.

1. ASP.NET Core MVC (Model-View-Controller)

<https://www.youtube.com/shorts/AMbMvgNEiMQ>

MVC is a design pattern used for creating dynamic web applications. It separates an application into three components:

Model

The **Model** represents the data and business logic of an application. It is responsible for storing, retrieving, and processing data. The model communicates with databases and ensures that business rules are applied correctly. For example, in a student management system, the Student model contains information such as Student ID, Name, and Course.

View

The **View** is the user interface of the application. It displays data to users and presents information in a readable format such as web pages, forms, or reports. The view receives data from the controller and shows it to the user. For example, a webpage displaying a list of students is a view.

Controller

The **Controller** acts as an intermediary between the Model and the View. It receives user requests, processes them, interacts with the model to retrieve or update data, and then selects the appropriate view to display the results. For example, when a user requests to see student details, the controller fetches the data from the model and sends it to the view for display.

This separation improves maintainability, testing, and code organization.

Example: An online sports event registration system where:

- Model stores participant information.
- View displays registration forms.
- Controller processes registration requests.

```
// Model
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; }
}
// Controller
public class StudentController : Controller
{
    public IActionResult Index()
    {
        return View();
    }
}
```

2. Razor Pages

Razor Pages is a page-based framework in ASP.NET Core used to build dynamic web applications with less code and a simpler structure than MVC. It combines the user interface (HTML) and page logic (C#) in a single page, making it easy for beginners to develop and maintain web applications. Each Razor Page consists of a .cshtml file for the user interface and a corresponding .cshtml.cs file, called the Page Model, which contains the page's processing logic.

Example

Index.cshtml

```
<h1>Welcome to ASP.NET Core Razor Pages</h1>
```

Index.cshtml.cs

using Microsoft.AspNetCore.Mvc.RazorPages;

```
public class IndexModel : PageModel
{
    public void OnGet()
    {
    }
}
```

When the application runs, the browser displays:

Welcome to ASP.NET Core Razor Pages

Razor Pages are particularly suitable for creating simple web applications, forms, and data-entry pages with a clean and organized structure.

3. Web API Framework

A **web API framework** is a collection of code libraries, routing tools, and utilities that streamlines building secure, scalable application programming interfaces over HTTP. It eliminates boilerplate code, allowing developers to focus strictly on business logic.

ASP.NET Core Web API is used to build RESTful services that allow communication between applications over the internet. APIs exchange data using formats such as JSON.

Features:

- Supports mobile and web applications.
- Lightweight and high performance.
- Easy integration with third-party systems.

Example: A sports analytics application providing player statistics to mobile apps through API endpoints.

4. Blazor Framework

Blazor is a modern web framework in ASP.NET Core that allows developers to build interactive web applications using **C# instead of JavaScript**. It enables the creation of dynamic user interfaces with reusable components and supports both client-side execution through WebAssembly and server-side execution through Blazor Server. Blazor helps developers use their existing .NET skills to create responsive and feature-rich web applications.

Example

Counter.razor

```
<h3>Counter</h3>
```

```

<p>Current count: @count</p>
<button @onclick="IncrementCount">Click Me</button>
@code {
    private int count = 0;
    private void IncrementCount()
    {
        count++;
    }
}

```

When the user clicks the button, the value of **count** increases and the updated count is displayed on the page without reloading it.

Blazor is suitable for building interactive applications such as dashboards, e-commerce websites, and single-page applications (SPAs) using C# and .NET.

2. Understanding the Utility Frameworks

Utility frameworks in **ASP.NET Core** are collections of libraries and services that provide common functionality required by applications. Instead of writing code for tasks such as database access, logging, configuration, dependency management, and security from scratch, developers can use these built-in frameworks. Utility frameworks simplify development, improve code reusability, and make applications easier to maintain and scale. ASP.NET Core provides several utility frameworks that work together with application frameworks like MVC, Razor Pages, and Web API.

1. Dependency Injection Framework

Dependency Injection (DI) is a built-in framework in ASP.NET Core that automatically provides required services and objects to classes. It reduces tight coupling between components and makes applications easier to test and maintain.

Example

```

public class HomeController : Controller
{
    private readonly ILogger<HomeController> _logger;

    public HomeController(ILogger<HomeController> logger)
    {
        _logger = logger;
    }

    public IActionResult Index()
    {
        _logger.LogInformation("Home page visited.");
        return View();
    }
}

```

In this example, the **ILogger** service is automatically supplied by the Dependency Injection framework.

2. Configuration Framework

The Configuration Framework is used to store and retrieve application settings. Configuration data is usually stored in the **appsettings.json** file and can include database connection strings, API keys, and application-specific settings.

Example

appsettings.json

```
{
  "MySettings": {
    "CompanyName": "ABC Technologies"
  }
}
```

Program.cs

```
var companyName = builder.Configuration["MySettings:CompanyName"];
Console.WriteLine(companyName);
```

Output

ABC Technologies

The application reads the value from the configuration file without hard-coding it into the program.

3. Logging Framework

The Logging Framework records application events, warnings, and errors. Logs help developers monitor application behavior and troubleshoot problems.

Example

```
public IActionResult About()
{
    _logger.LogInformation("About page accessed.");
    return View();
}
```

When the About page is opened, the framework records the message in the log file or console.

4. Entity Framework Core

Entity Framework Core (EF Core) is an Object-Relational Mapper (ORM) that allows developers to interact with databases using C# objects instead of SQL queries. It simplifies database operations such as inserting, updating, deleting, and retrieving data.

Example

Student Class

```
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; }
}
```

Adding Data

```
context.Students.Add(new Student
{
    Name = "Rahul"
});

context.SaveChanges();
```

This code inserts a new student record into the database.

5. Middleware Framework

Middleware components process HTTP requests and responses. They form a pipeline through which every request passes before reaching the application.

Example

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.UseHttpsRedirection();

app.MapGet("/", () => "Welcome to ASP.NET Core");

app.Run();
```

Here, **UseHttpsRedirection()** is middleware that redirects HTTP requests to HTTPS for improved security.

6. Authentication and Authorization Framework

These frameworks provide security by verifying users (authentication) and controlling access to resources (authorization).

Example

```
[Authorize]
public IActionResult Dashboard()
{
    return View();
}
```

Only authenticated users can access the Dashboard page.

7. Caching Framework

Caching improves application performance by temporarily storing frequently used data in memory, reducing database access and response time.

Example

```
memoryCache.Set("Message", "Hello ASP.NET Core");
```

Later, the stored value can be retrieved quickly without recalculating or fetching it again.

Utility Framework	Purpose	Example
Dependency Injection	Provides required services automatically	ILogger injection
Configuration Framework	Manages application settings	appsettings.json
Logging Framework	Records information and errors	_logger.LogInformation()
Entity Framework Core	Simplifies database access	context.Students.Add()
Middleware Framework	Processes HTTP requests and responses	UseHttpsRedirection()
Authentication and Authorization	Secures applications	[Authorize] attribute
Caching Framework	Improves performance	memoryCache.Set()

3. Understanding the ASP.NET Core Platform

The ASP.NET Core platform consists of a collection of frameworks, libraries, and tools that help developers build and run web applications. It provides everything needed to handle web requests, process business logic, access databases, manage security, and generate responses for users. ASP.NET Core supports different development approaches such as MVC, Razor Pages, Blazor, and Web API.

Example

A college can develop a **Student Management System** using ASP.NET Core to:

- Manage student records
- Display attendance information
- Generate examination results
- Provide online registration facilities

Key Features of ASP.NET Core

1. Cross-Platform Support

ASP.NET Core applications can run on multiple operating systems, including Windows, Linux, and macOS. This gives organizations flexibility in choosing their deployment environment.

Example

A web application developed on Windows can be deployed on a Linux server without changing the code.

2. Open Source

ASP.NET Core is open source, meaning developers can view, modify, and contribute to its source code. This encourages community support and continuous improvement.

Example

Developers can access the ASP.NET Core source code and learn how different framework components work internally.

3. High Performance

ASP.NET Core is optimized for speed and efficient resource utilization. It can handle a large number of requests while maintaining good performance.

Example

An e-commerce website can serve thousands of users simultaneously without significant delays.

4. Built-in Dependency Injection

Dependency Injection (DI) is included in ASP.NET Core by default. It helps manage application services and reduces tight coupling between components.

Example

```
public class HomeController : Controller
{
    private readonly ILogger<HomeController> _logger;

    public HomeController(ILogger<HomeController> logger)
    {
        _logger = logger;
    }
}
```

The framework automatically provides the logger service to the controller.

5. Middleware Pipeline

ASP.NET Core uses middleware components to process HTTP requests and responses. Each middleware performs a specific task before passing control to the next component.

Example

```
app.UseHttpsRedirection();
app.UseRouting();
app.UseAuthorization();
```

These middleware components handle HTTPS redirection, routing, and authorization.

6. Built-in Security

ASP.NET Core includes features for authentication, authorization, data protection, and secure communication.

Example

```
[Authorize]
public IActionResult Dashboard()
{
    return View();
}
```

Only authenticated users can access the Dashboard page.

ASP.NET Core Application Structure

An ASP.NET Core application consists of several important files and folders:

Program.cs

The entry point of the application where services and middleware are configured.

appsettings.json

Stores application configuration settings such as connection strings and application options.

Controllers

Contain application logic and handle user requests.

Models

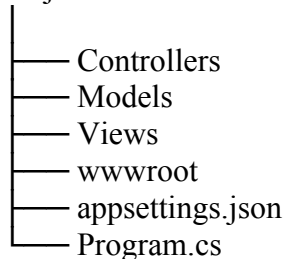
Represent the application's data.

Views

Display information to users in MVC applications.

Example Structure

Project



Creating a Simple ASP.NET Core Application

Program.cs

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
app.MapGet("/", () => "Welcome to ASP.NET Core!");
app.Run();
```

Output

Welcome to ASP.NET Core!

When the application runs, the browser displays the message **"Welcome to ASP.NET Core!"**.

Advantages of ASP.NET Core

- Fast and high-performance framework
- Supports Windows, Linux, and macOS
- Open-source and community-driven
- Built-in security features
- Easy integration with databases
- Supports cloud deployment

- Scalable for enterprise applications
- Multiple development models (MVC, Razor Pages, Blazor, Web API)

Example

A university portal can use ASP.NET Core to manage admissions, attendance, examinations, and student records through a single web application.

Using the Development Tools in ASP.NET Core

Reference: Freeman, A. (2022). *Pro ASP.NET Core 6* (9th ed.). Apress.

<https://www.youtube.com/watch?v=NwzXh95S9ik>

Introduction

ASP.NET Core provides a rich set of development tools that simplify the process of creating, building, testing, debugging, and maintaining web applications. Developers can use **Visual Studio**, **Visual Studio Code**, and the **.NET Command Line Interface (CLI)** to develop applications efficiently. These tools help automate many tasks and improve productivity.

1. Creating ASP.NET Core Projects

Introduction

An ASP.NET Core project is a collection of files, folders, source code, and configuration settings that together form a web application. Projects can be created using Visual Studio or the .NET CLI.

Creating a Project Using Visual Studio

Step 1: Start Visual Studio

Open Visual Studio and select:

Create a new project

Step 2: Select a Project Template

Common templates include:

- ASP.NET Core Web App
- ASP.NET Core Web API
- ASP.NET Core Web App (Model-View-Controller)
- Razor Class Library

Choose:

ASP.NET Core Web App (Model-View-Controller)

Click **Next**.

Step 3: Configure the Project

Specify:

- Project Name
- Location
- Solution Name

Example:

StudentManagement

Click **Next**.

Step 4: Additional Settings

Select:

- Framework: .NET 6.0
- Authentication Type: None
- Enable HTTPS

Click:

Create

Visual Studio generates the project automatically.

Creating Projects Using .NET CLI

Create a project:

```
dotnet new mvc -n StudentManagement
```

Move to project folder:

```
cd StudentManagement
```

Run the application:

```
dotnet run
```

Project Structure

StudentManagement

```
|
|— Controllers
|— Models
|— Views
|— wwwroot
|— Properties
|— appsettings.json
|— Program.cs
```

└─ StudentManagement.csproj

Purpose of Main Components

Component	Purpose
Controllers	Process requests
Models	Store application data
Views	User interface
wwwroot	Static files
Program.cs	Entry point
appsettings.json	Configuration settings
.csproj	Project information

2. Adding Code and Content to Projects

Introduction

After creating a project, developers add source code, web pages, stylesheets, images, and other resources.

Adding C# Classes

Example:

Create a file named **Student.cs** inside Models folder.

```
namespace StudentManagement.Models
{
    public class Student
    {
        public int Id { get; set; }

        public string Name { get; set; }

        public int Age { get; set; }
    }
}
```

This class represents student data.

Adding Controllers

Create HomeController.cs inside Controllers folder.

```
using Microsoft.AspNetCore.Mvc;

namespace StudentManagement.Controllers
{
    public class HomeController : Controller
```

```

    {
        public IActionResult Index()
        {
            return View();
        }
    }
}

```

Controllers handle incoming requests.

Adding Views

Create:

Views/Home/Index.cshtml

Example:

```
<h1>Welcome to ASP.NET Core</h1>
```

```
<p>Student Management System</p>
```

The browser displays this page when the Index action is called.

Adding Static Content

Static files are stored inside **wwwroot**.

Examples:

wwwroot/css/site.css

wwwroot/js/site.js

wwwroot/images/logo.png

Example CSS

```

body
{
    background-color: lightblue;
}

```

Adding Configuration Settings

Inside appsettings.json:

```

{
  "ApplicationName": "Student Management System"
}

```

These settings can be read by the application.

Adding Existing Files

Visual Studio provides:

Project → Add Existing Item

This allows adding:

- Images
- CSS files
- JavaScript files
- C# files

3. Building and Running Projects

Introduction

Building converts source code into executable files. Running starts the application and makes it available to users.

Build Process

The compiler performs:

1. Syntax checking.
2. Reference verification.
3. Compilation into Intermediate Language (IL).
4. Creation of DLL files.

Build Using Visual Studio

Select:

Build → Build Solution

Shortcut:

Ctrl + Shift + B

Output appears in:

bin/Debug/net6.0

Build Using CLI

dotnet build

Example output:

Build succeeded.

Running the Application

Using Visual Studio

Press:

F5

or

Ctrl + F5

The browser automatically opens.

Using Command Line

dotnet run

Output:

Now listening on:

https://localhost:5001

Open:

https://localhost:5001

Rebuilding a Project

Sometimes old files must be removed and recreated.

Visual Studio:

Build → Rebuild Solution

CLI:

dotnet clean

dotnet build

Publishing a Project

To deploy an application:

Visual Studio:

Publish

CLI:

dotnet publish

The published files are stored in:

bin/Release/net6.0/publish

4. Managing Packages

Introduction

ASP.NET Core uses NuGet packages to provide additional libraries and functionality.

Examples:

- Entity Framework Core
- Authentication libraries
- Logging libraries

- JSON serialization libraries

What is a Package?

A package contains:

- DLL files
- Metadata
- Dependencies
- Version information

Installing Packages Using Visual Studio

Right-click project:

Manage NuGet Packages

Search:

Microsoft.EntityFrameworkCore.SqlServer

Click:

Install

Installing Packages Using CLI

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
```

Viewing Installed Packages

Visual Studio:

Dependencies → Packages

CLI:

```
dotnet list package
```

Updating Packages

Visual Studio:

Manage NuGet Packages → Updates

CLI:

```
dotnet add package PackageName --version 7.0.0
```

Removing Packages

Visual Studio:

Manage NuGet Packages → Uninstall

CLI:

```
dotnet remove package Microsoft.EntityFrameworkCore.SqlServer
```

Package Reference Example

Inside .csproj file:

```
<ItemGroup>
  <PackageReference
    Include="Microsoft.EntityFrameworkCore.SqlServer"
    Version="6.0.0" />
</ItemGroup>
```

Package Restore

Downloads missing packages.

CLI:

```
dotnet restore
```

Visual Studio restores packages automatically.

5. Debugging Projects

Debugging is the process of identifying and correcting errors in a program. Visual Studio provides powerful debugging tools that allow developers to monitor program execution.

Types of Errors

Syntax Errors

Detected during compilation.

Example:

```
int x =
```

Compiler reports an error.

Runtime Errors

Occur while the program executes.

Example:

```
int x = 10;
```

```
int y = 0;
```

```
int z = x / y;
```

Produces:

DivideByZeroException

Logical Errors

Program executes but produces incorrect results.

Example:

```
int total = 10 + 20 * 5;
```

Incorrect assumptions may lead to wrong output.

Breakpoints

Breakpoints pause program execution.

Set breakpoint:

- Click left margin beside code.
- Press:

F9

Example:

```
int a = 10;  
int b = 20;  
int sum = a + b; // Breakpoint here
```

Execution stops before calculating sum.

Step-by-Step Execution

Step Into (F11)

Executes one statement at a time and enters functions.

Example:

```
Calculate();
```

Debugger moves inside Calculate().

Step Over (F10)

Executes a statement without entering functions.

Step Out (Shift + F11)

Completes current function and returns to the caller.

Watch Window

Allows monitoring variables.

Example:

```
int marks = 80;
```

Watch Window displays:

```
marks = 80
```

Locals Window

Displays all variables currently in scope.

Example:

```
name = "Rahul"  
age = 20
```

Immediate Window

Used for executing statements during debugging.

Example:

```
a+b
```

Output:

```
30
```

Exception Handling During Debugging

Example:

```
try  
{  
    int x = 10;  
    int y = 0;  
  
    int z = x / y;  
}  
catch(Exception ex)
```

```
{  
    Console.WriteLine(ex.Message);  
}
```

Output:

Attempted to divide by zero.

Debugging ASP.NET Core Applications

Start debugging:

F5

Visual Studio launches:

- Kestrel Web Server
- Browser

Breakpoints can be placed in:

- Controllers
- Models
- Middleware
- Services

Example:

```
public IActionResult Index()  
{  
    string message = "Welcome";  
  
    return View();  
}
```

A breakpoint on `message` allows inspecting the value before returning the view.

Advantages of Development Tools

- Faster application development.
- Automatic project creation.
- Easy package management.
- Efficient debugging.
- Better code organization.
- Simplified deployment.
- Support for collaboration and maintenance.

Development tools in ASP.NET Core provide a complete environment for creating, building, running, managing, and debugging applications. Visual Studio and the .NET CLI enable developers to efficiently create projects, add code and content, install NuGet packages, and

identify errors using advanced debugging features. These tools significantly improve productivity and simplify modern web application development.